

MODULE 11 – APPLICATION DEBUGGING

1. Introduction

Application debugging is the process of identifying, analyzing, and fixing errors in a software application. In React applications, debugging helps developers locate issues related to components, state management, props, rendering, JavaScript logic, and browser behavior. Popular debugging tools include **Chrome DevTools**, **React Developer Tools**, and **Visual Studio Code Debugger**.

2. Debugging in Visual Studio Code

Visual Studio Code provides a built-in debugger that allows developers to pause program execution, inspect variables, and analyze application behavior.

Features of Visual Studio Code Debugger

- Setting breakpoints
- Inspecting variables
- Watching expressions
- Viewing call stack
- Step Into, Step Over, and Step Out execution
- Debug Console
- Source code navigation

3. Setting Breakpoints

A **breakpoint** is a marker placed in the source code where program execution pauses during debugging.

Purpose of Breakpoints

- Pause program execution.
- Inspect variable values.
- Identify logical errors.
- Analyze application flow.

Types of Breakpoints

- Line Breakpoint
- Conditional Breakpoint
- Function Breakpoint
- Logpoint

4. Inspecting Variables

During debugging, Visual Studio Code allows developers to inspect variable values.

Watch Window

The Watch Window monitors selected variables or expressions during program execution.

Locals Window

The Locals Window displays all local variables available in the current execution context.

Benefits

- Monitor variable changes.
- Verify program logic.
- Detect incorrect values.
- Simplify debugging.

5. Stepping Through Code

Visual Studio Code provides different execution controls.

Step Into (F11)

Moves inside a function to execute it line by line.

Step Over (F10)

Executes the current line without entering function calls.

Step Out (Shift + F11)

Completes the current function and returns to the calling function.

Continue (F5)

Resumes program execution until the next breakpoint.

6. Debugging React Applications with Chrome DevTools

Chrome DevTools is a browser-based debugging tool used to inspect, analyze, and debug web applications.

It provides features for:

- DOM inspection
- CSS editing
- JavaScript debugging
- Network analysis
- Performance monitoring
- React component inspection

7. Setting Up the Environment

Before debugging a React application:

- Install Node.js
- Install Visual Studio Code
- Install Google Chrome
- Install React Developer Tools Extension
- Install project dependencies using:

```
npm install
```

Start the application using:

```
npm start
```

or

```
npm run dev
```

8. Launching the Application

Run the React application.

Example:

```
npm start
```

The application opens in the browser, usually at:

```
http://localhost:3000
```

or

```
http://localhost:5173
```

9. Opening Chrome DevTools

Chrome DevTools can be opened using:

- Press **F12**
- Press **Ctrl + Shift + I**
- Right-click the webpage and select **Inspect**

The main DevTools panels include:

- Elements
- Console
- Sources
- Network
- Performance
- Memory
- Application

10. Inspecting Elements

The **Elements** panel displays the HTML structure (DOM) and applied CSS styles.

Developers can:

- Inspect HTML elements
- Modify CSS properties
- View applied styles
- Check layout information
- Debug responsive design

11. Inspecting the DOM Tree

The DOM Tree represents the hierarchical structure of HTML elements.

Using the DOM Tree, developers can:

- Locate components
- Verify rendered elements
- Check nested structures
- Identify missing or incorrect elements

12. Debugging JavaScript

The **Sources** panel is used to debug JavaScript code.

Features include:

- Set breakpoints
- Pause execution
- Inspect variables
- View Call Stack
- Step through code
- Resume execution

Developers can easily identify logical and runtime errors.

13. Source Maps

Source Maps map compiled JavaScript back to the original source files (such as TypeScript or JSX).

Benefits:

- Debug original source code
- Easier error tracing
- Better debugging experience
- Improved readability

14. Debugging TypeScript Code

When debugging TypeScript applications:

- Source Maps convert JavaScript back to TypeScript.
- Breakpoints can be placed directly in `.ts` or `.tsx` files.
- Variables can be inspected in their original TypeScript form.

15. React Developer Tools

React Developer Tools is a browser extension used to inspect React applications.

Features include:

- View component hierarchy
- Inspect component props
- Inspect component state
- Analyze Hooks
- Identify unnecessary re-renders
- Edit props and state during debugging

16. Debugging React Components

React components can be debugged by:

- Setting breakpoints in component code.
- Inspecting Props.
- Inspecting State.
- Monitoring Hooks.
- Checking component rendering.

17. Inspecting Application State

Application state stores dynamic data used by React components.

Developers can:

- View current state values.
- Verify state updates.
- Identify incorrect state changes.
- Track state transitions.

18. Common Frontend Issues

Common issues found during debugging include:

- Undefined variables
- Incorrect props
- State update errors
- Infinite rendering loops
- Missing dependencies
- Event handling errors
- API response issues
- CSS layout problems

19. Effective Debugging Techniques

- Read error messages carefully.
- Use breakpoints instead of multiple console logs.
- Inspect variables frequently.
- Debug one issue at a time.
- Verify API responses.
- Check component state and props.
- Test edge cases.
- Use React Developer Tools for component inspection.

20. Improving Application Performance

Debugging also helps improve performance by identifying:

- Unnecessary component re-renders
- Memory leaks
- Slow rendering
- Large JavaScript bundles
- Expensive computations
- Unoptimized state updates

Performance optimization leads to faster and more reliable React applications.

21. Conclusion

Application debugging is an essential part of React development. Tools such as Visual Studio Code Debugger, Chrome DevTools, and React Developer Tools help developers efficiently identify and resolve issues. Proper debugging techniques improve application quality, maintainability, reliability, and overall performance.